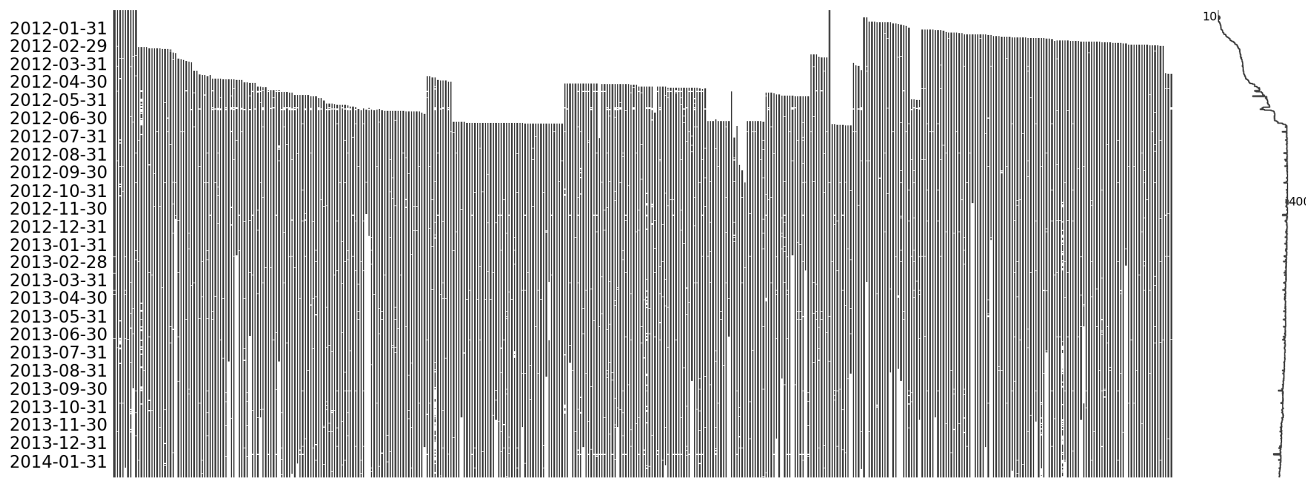
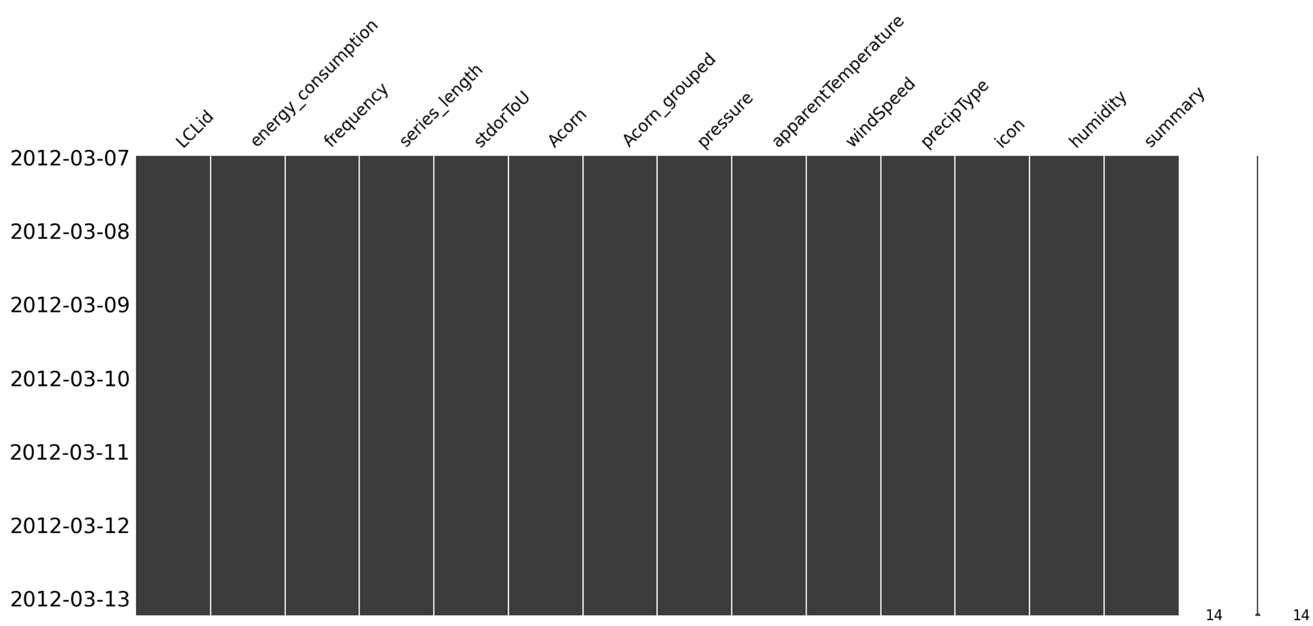
Preprocessing Time-Series Data

Missing Data and Imputation Techniques

```
#Pivot data to set index as datetime and different time series along columns
```

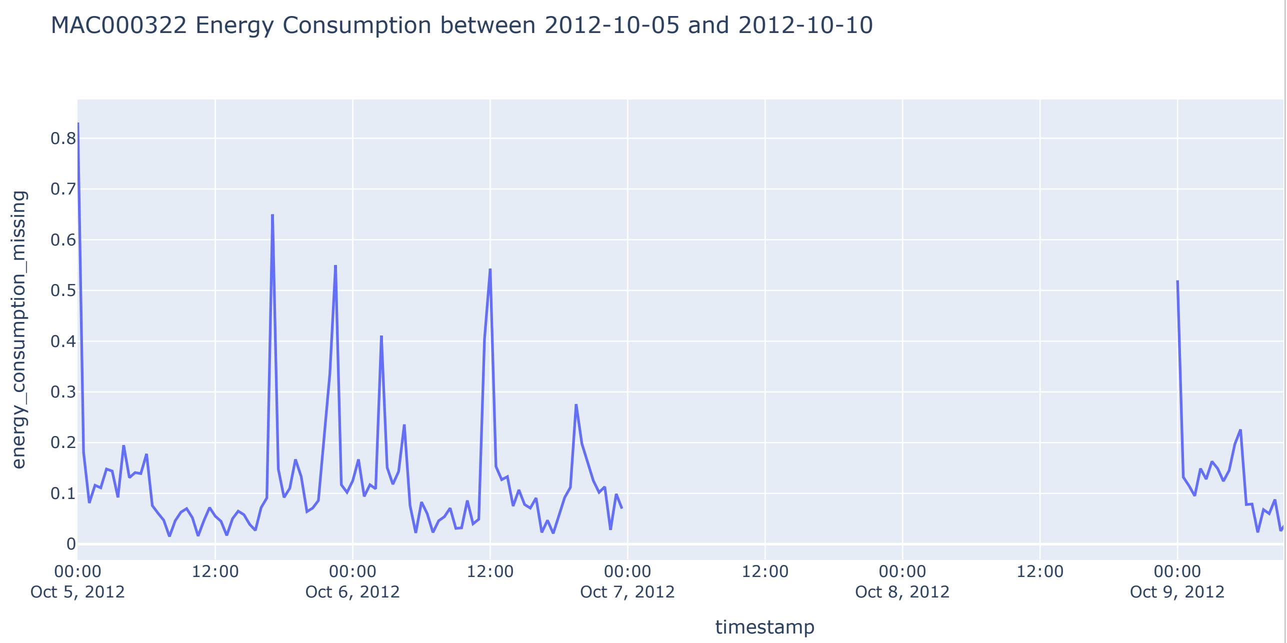



```
# take a single timeseries block from the data
```



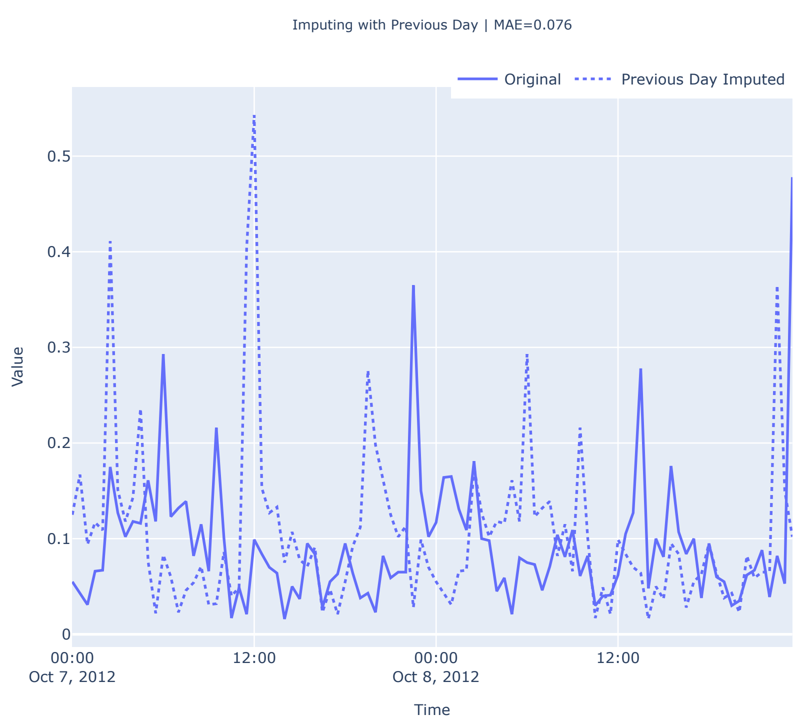
Create a missing data section to impoute the data and assess how the imputation technique works.

```
''' MAC000322 Energy Consumption '''
```



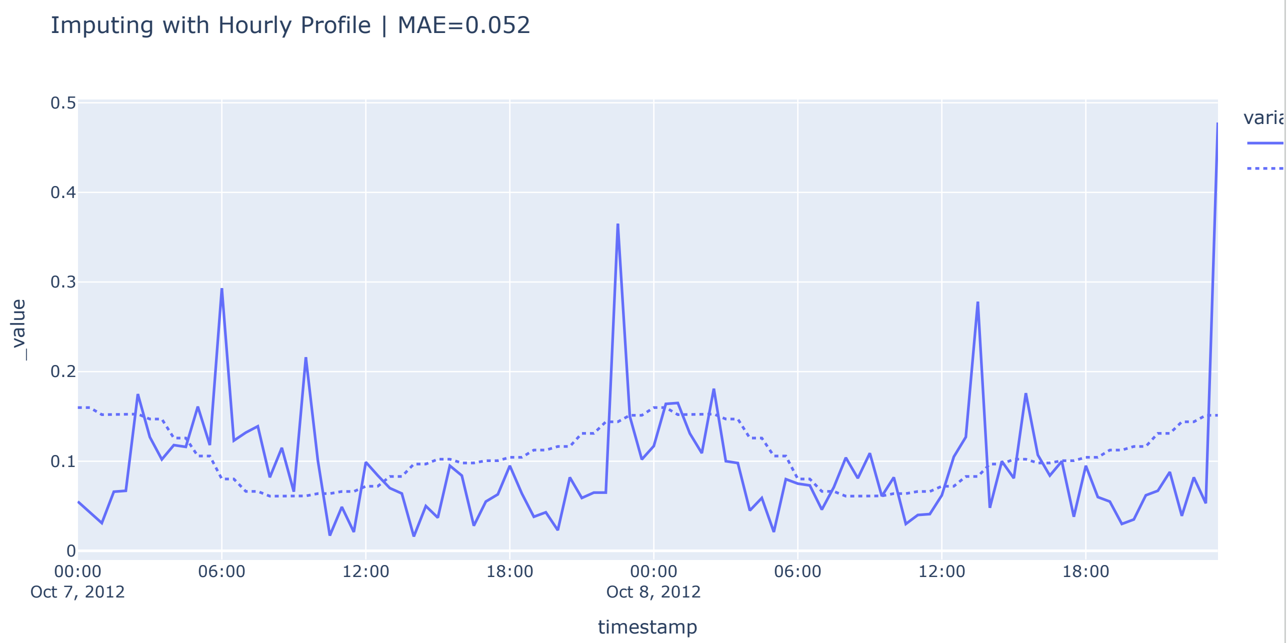
Previous day

```
''' Imputing with Previous Day '''
```



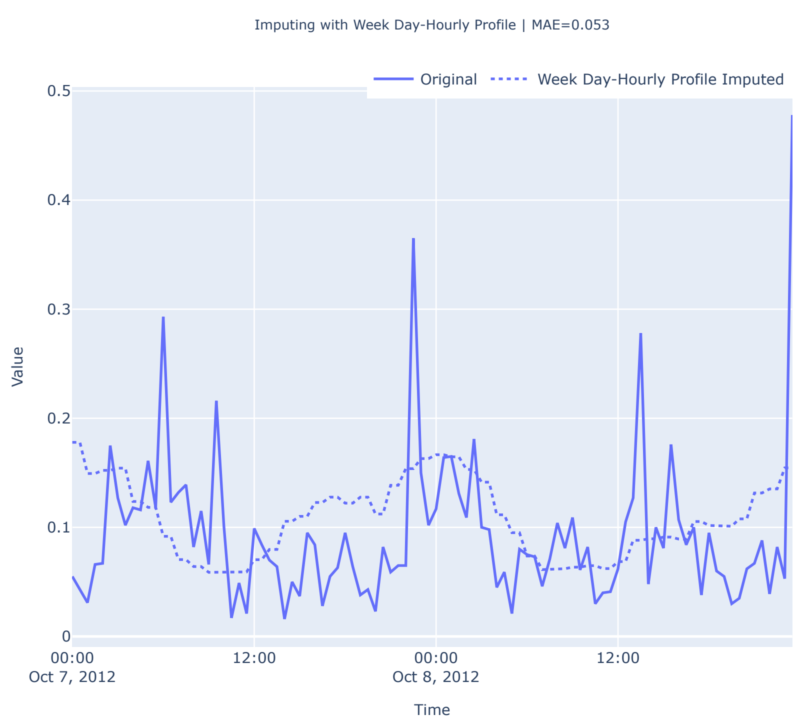
Hourly profile

```
''' Imputing with Hourly Profile '''
```

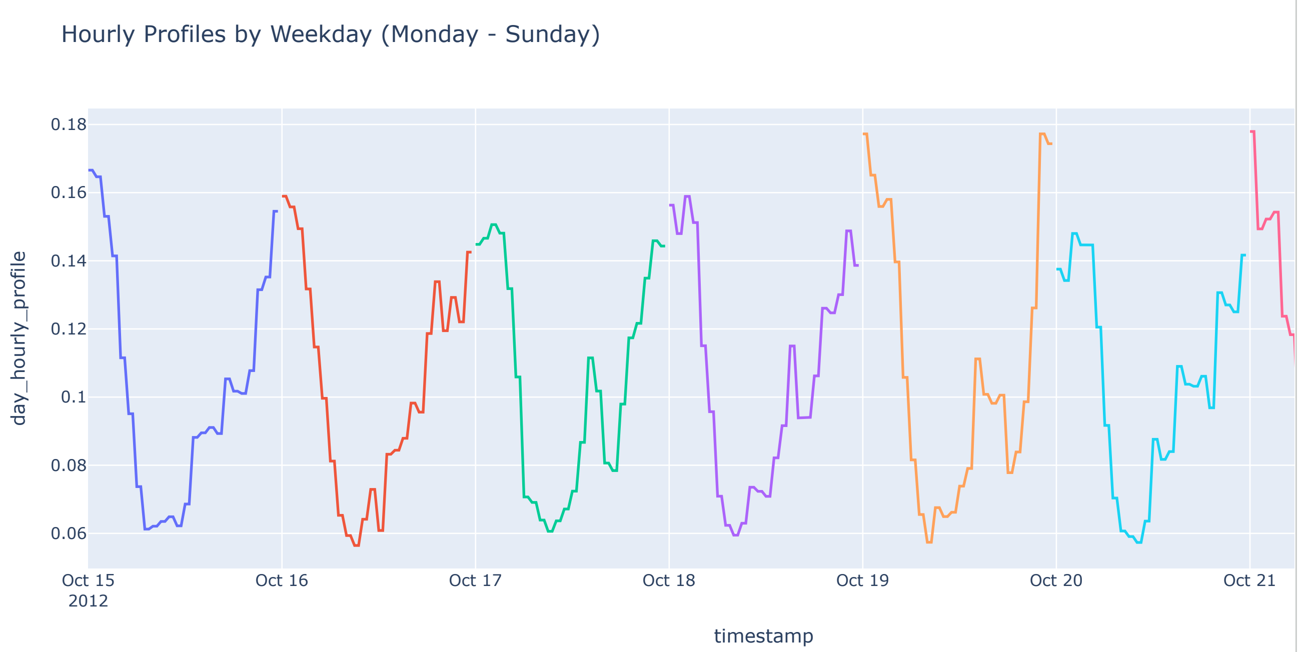


Weekly hourly profile

```
''' Imputing with Week Day-Hourly Profile '''
```

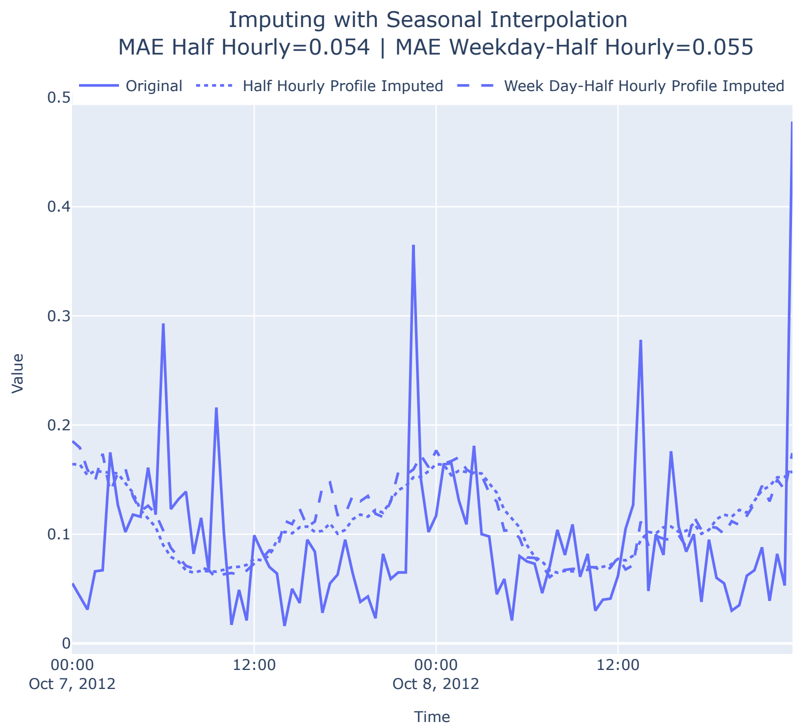



```
''' Hourly Profiles by Weekday (Monday - Sunday) '''
```



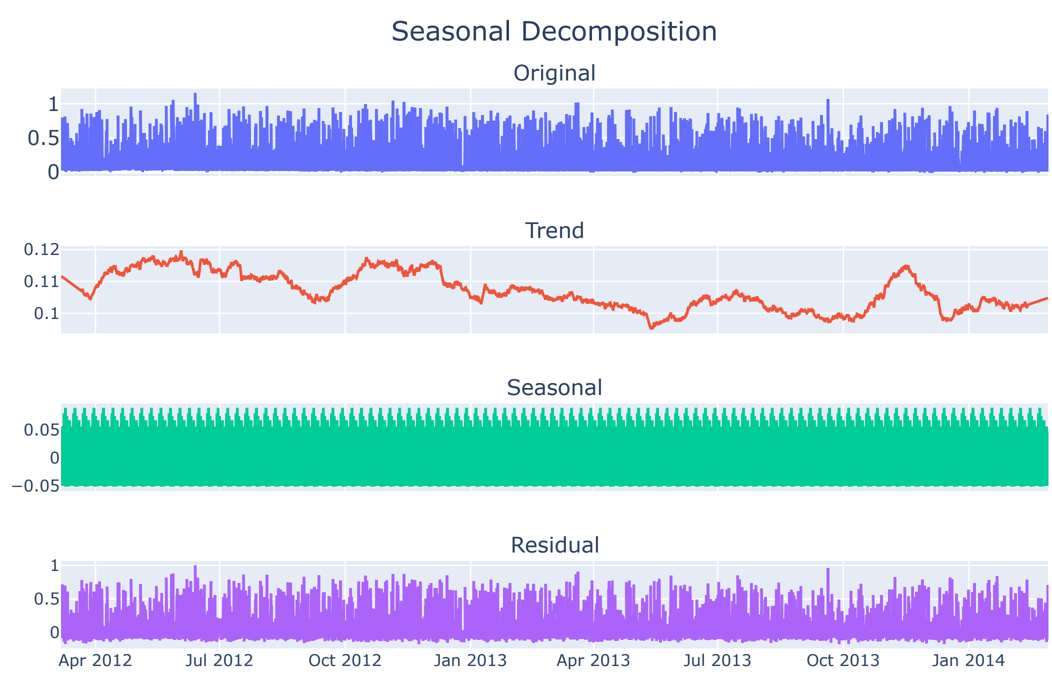
Seasonal Interpolation

```
''' Imputing with Seasonal Interpolation '''
```

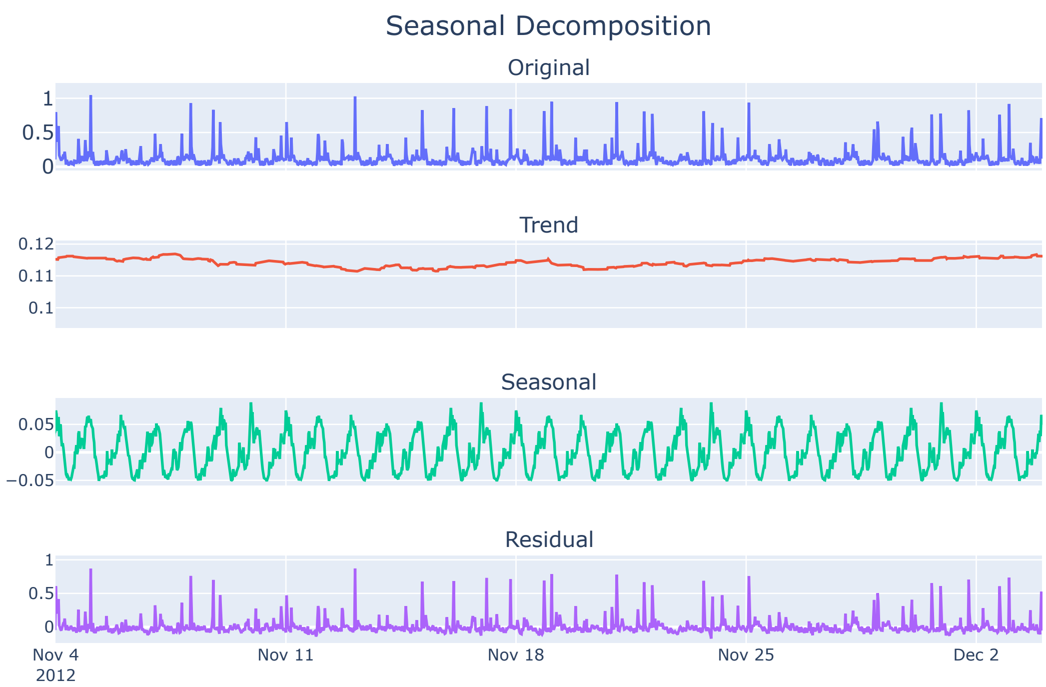


Seasonal Decomposition

```
#Does not support misssing values, so using imputed ts instead
#res = seasonal_decompose(ts, period=7*48, model="additive", #extrapolate_trend="freq", filt=np.repeat(1/(30*48), 30
```

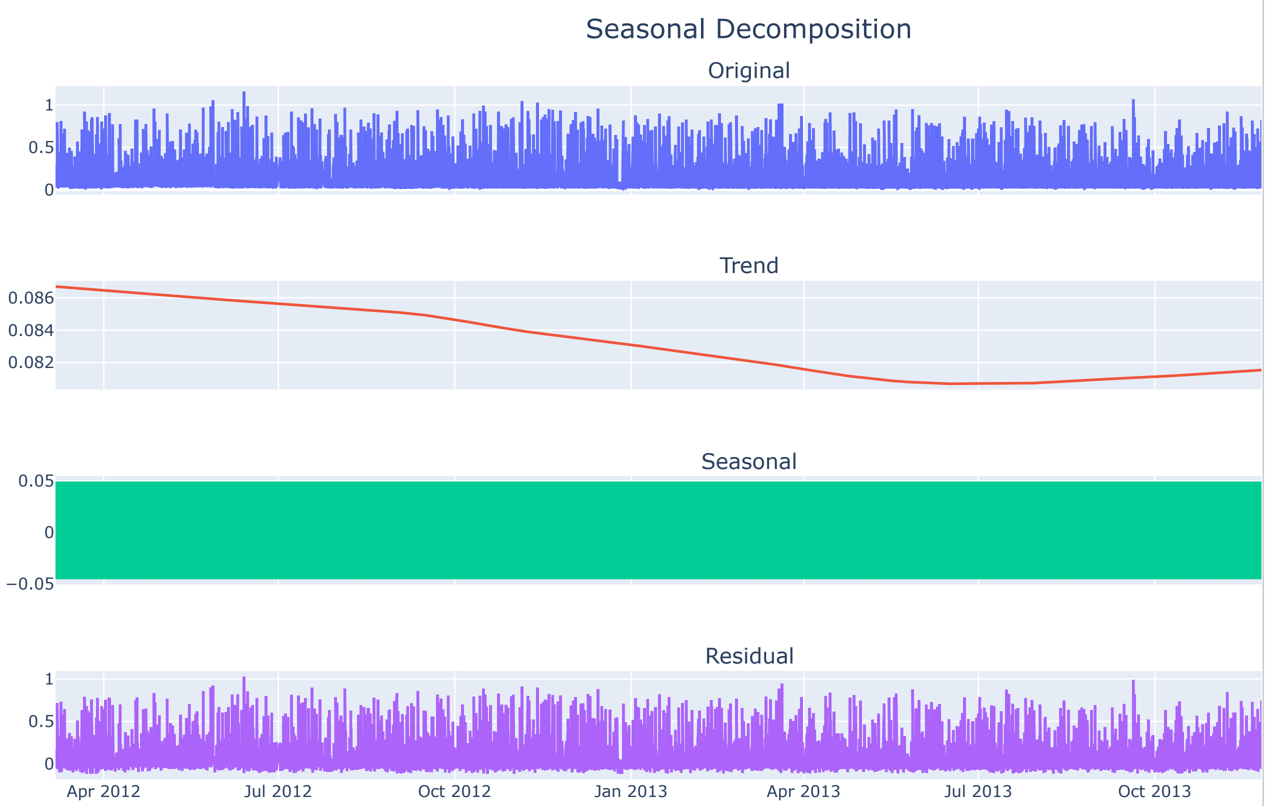



```
#Let's zoom in on a few days to better see the seasonality extracted
#fig.update_xaxes(type="date", range=["2012-11-4", "2012-12-4"])
```

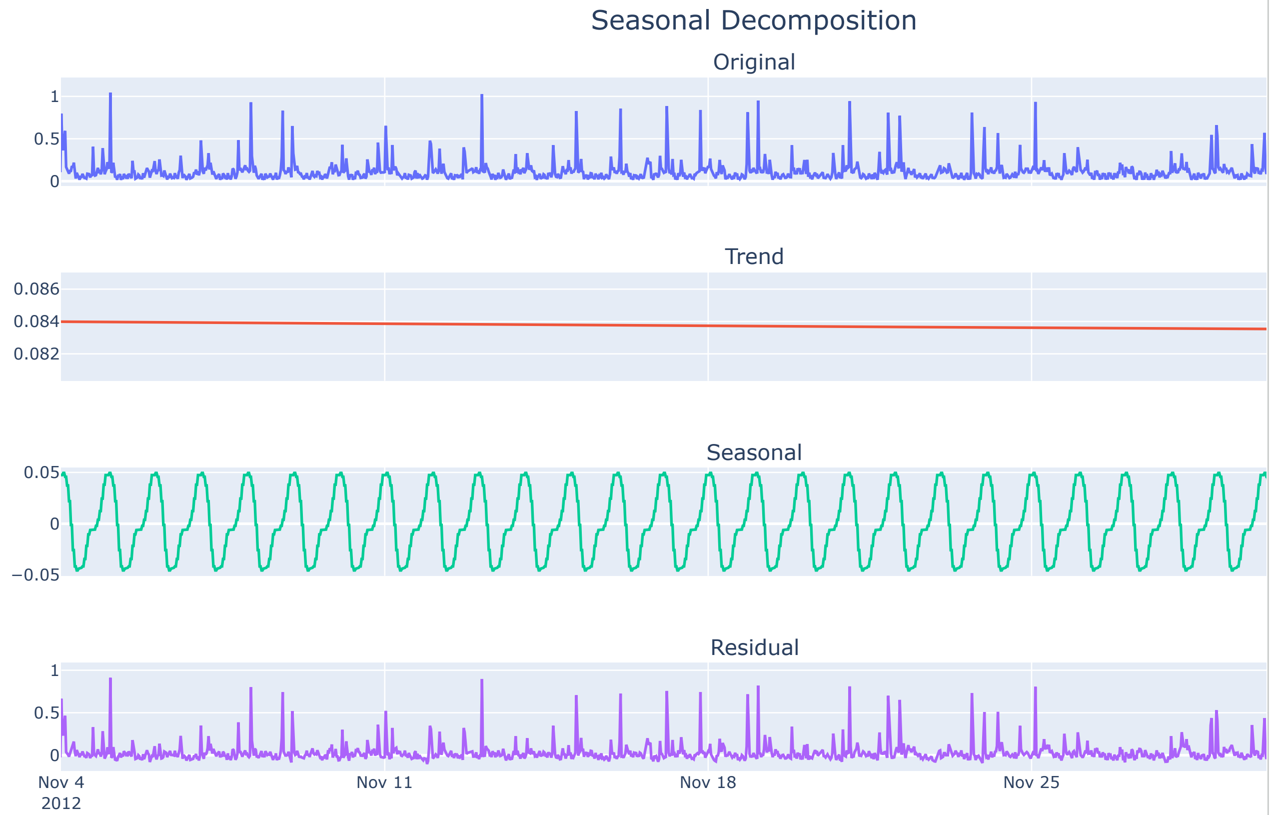


Seasonality and Trend Decomposition using Loess (STL)

```
#Supports missing values and expects a series or dataframe with datetime index
stl = STL(seasonality_period=7*48, model = "additive")
res_new = stl.fit(ts_df.energy_consumption)
```

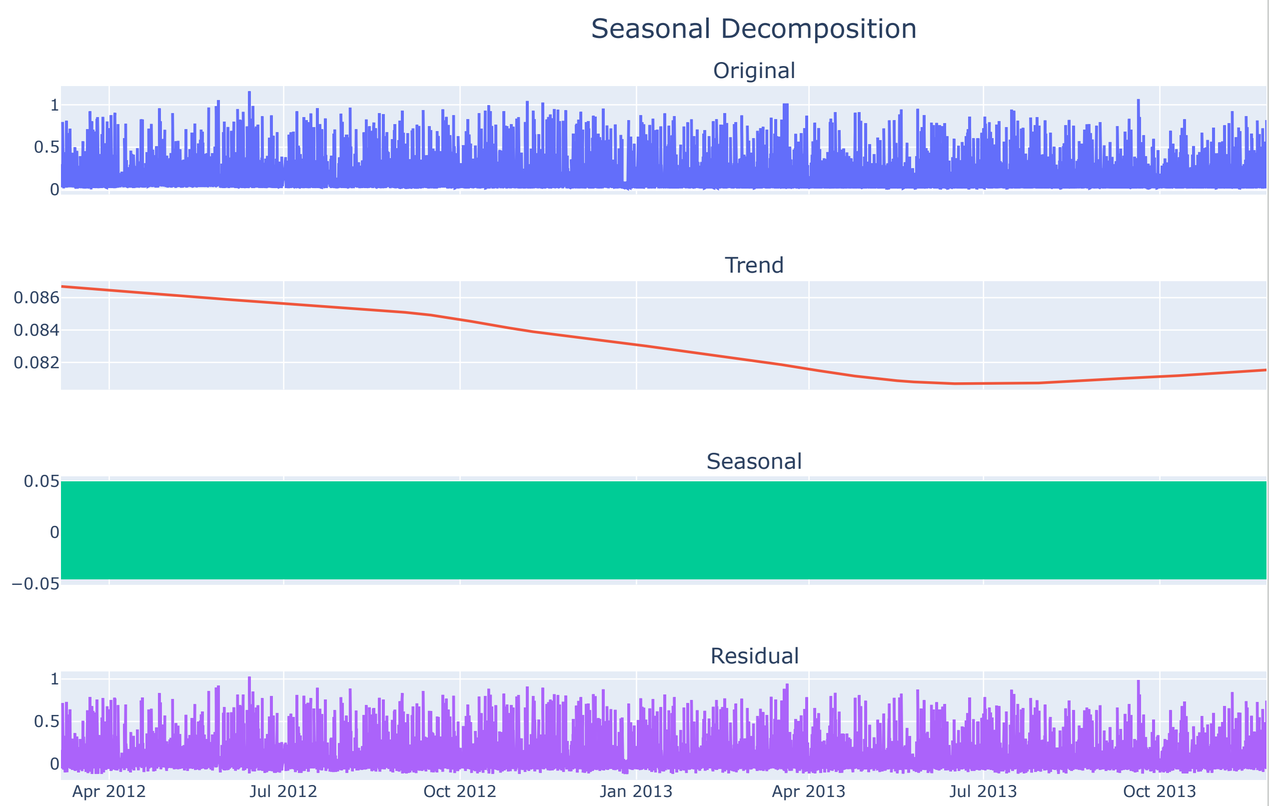



```
#Let's zoom in on a few days to better see the seasonality extracted
#fig.update_xaxes(type="date", range=["2012-11-4", "2012-12-4"])
```

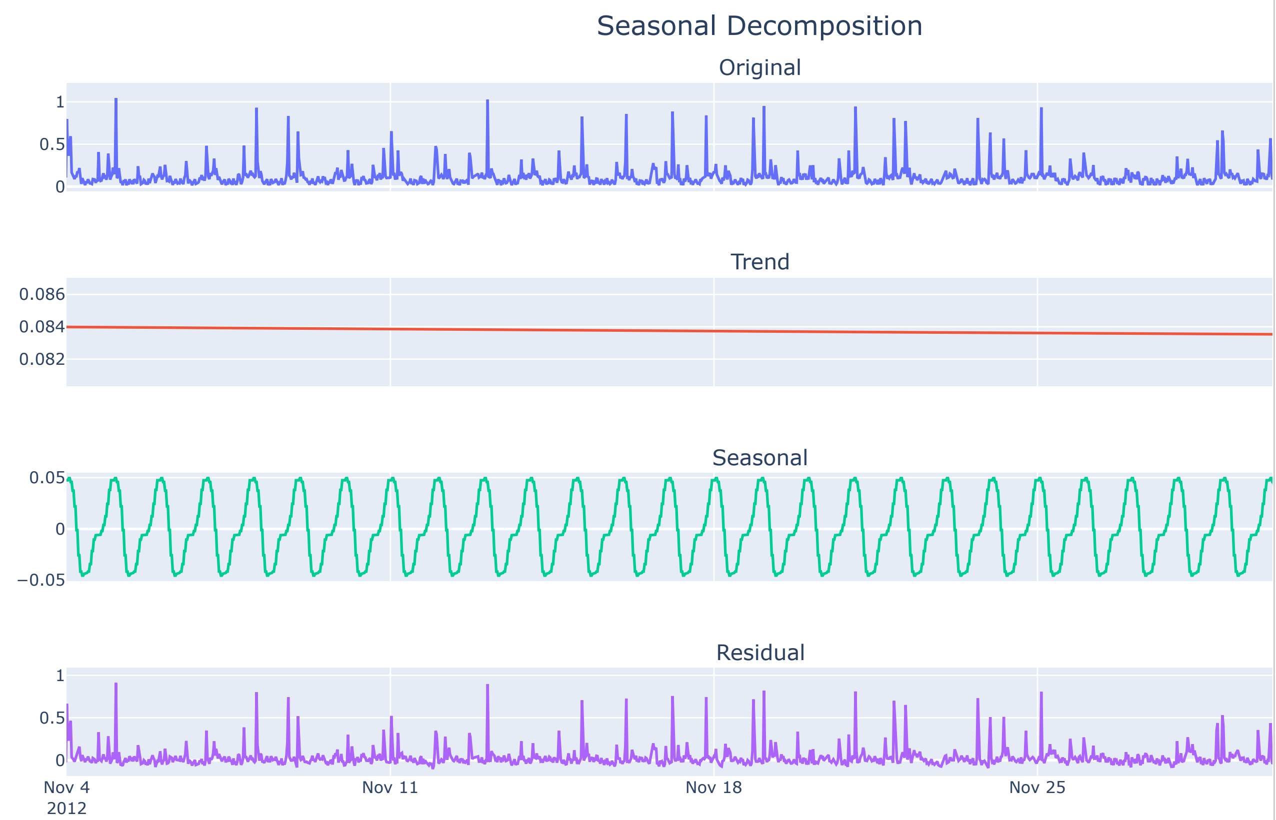


Seasonality and Trend Decomposition using Loess and Fourier Terms (Fourier Decomposition)

```
#Doesn't support missing values, and expects a series or datafeame with datetime index
stl = FourierDecomposition(
    seasonality_period="hour", model = "additive", n_fourier_terms=5)
res_new = stl.fit(pd.Series(ts.squeeze(), index=ts_df.index))
```

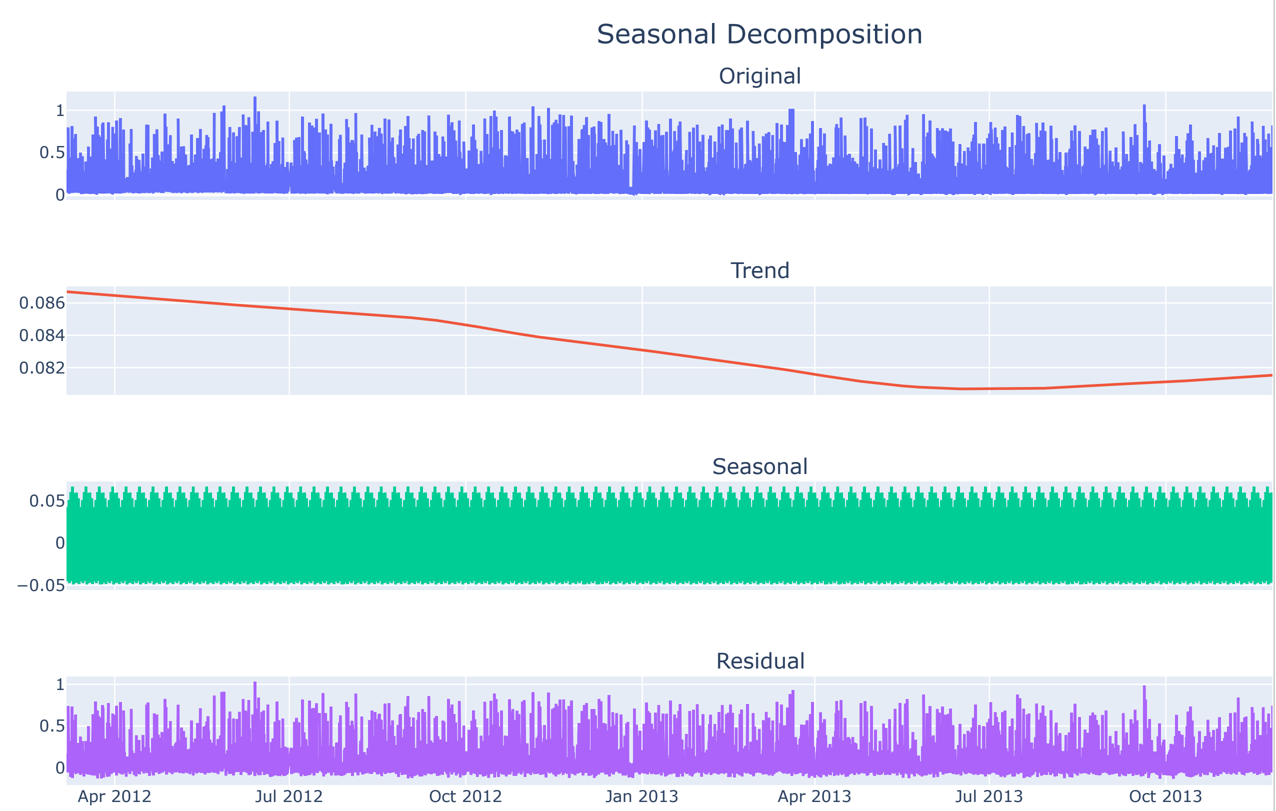



```
#fig.update_xaxes(type="date", range=["2012-11-4", "2012-12-4"])
```

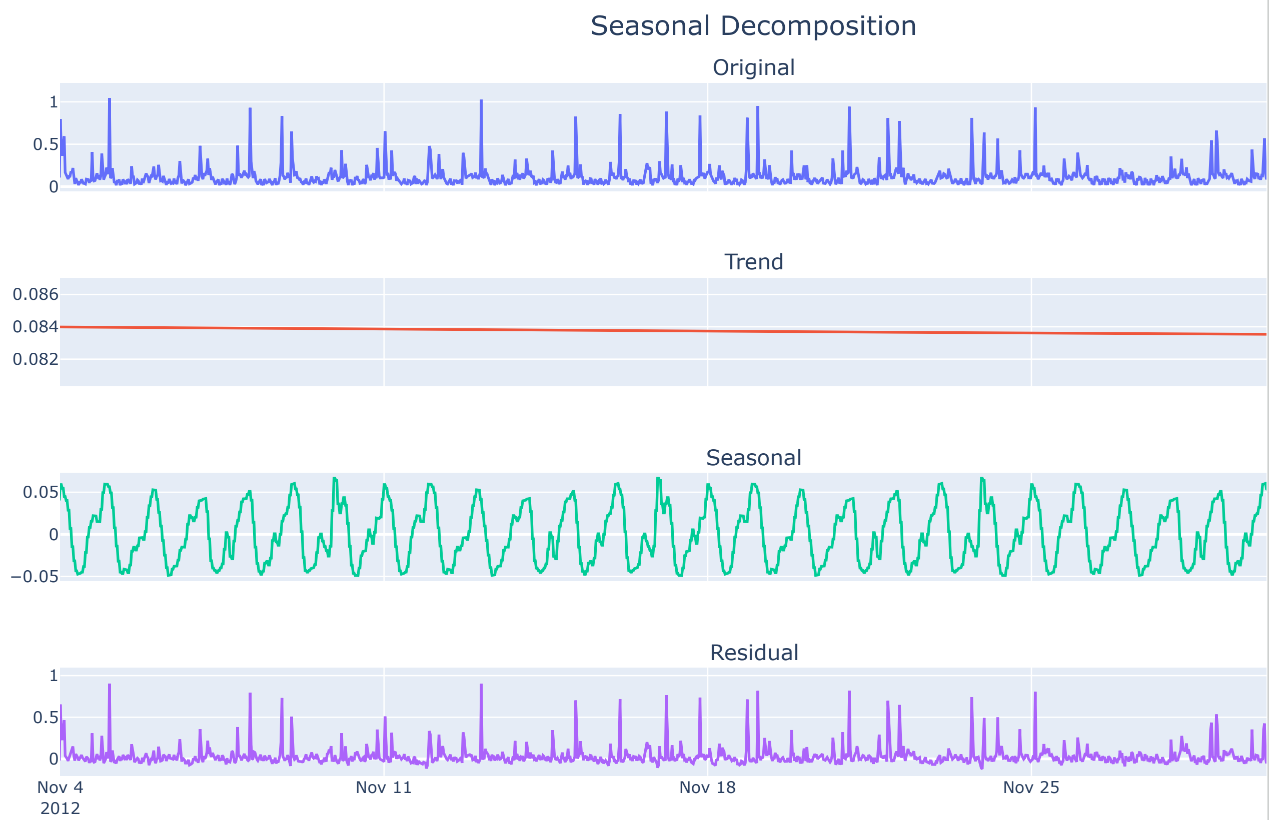


Custom Seasonality

```
stl = FourierDecomposition(model = "additive", n_fourier_terms=50)
res_new = stl.fit(pd.Series(ts, index=ts_df.index), seasonality=seasonality)
```



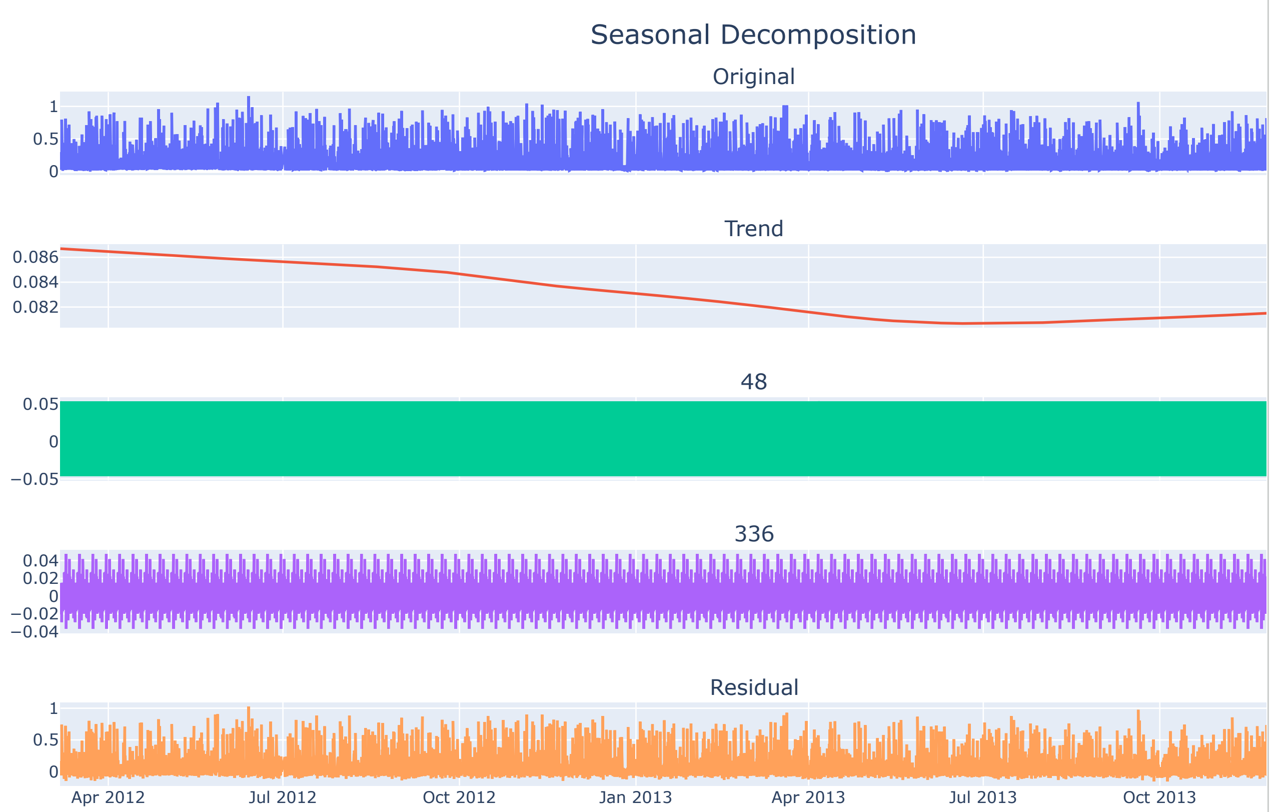

```
#fig.update_xaxes(type="date", range=["2012-11-4", "2012-12-4"])
```



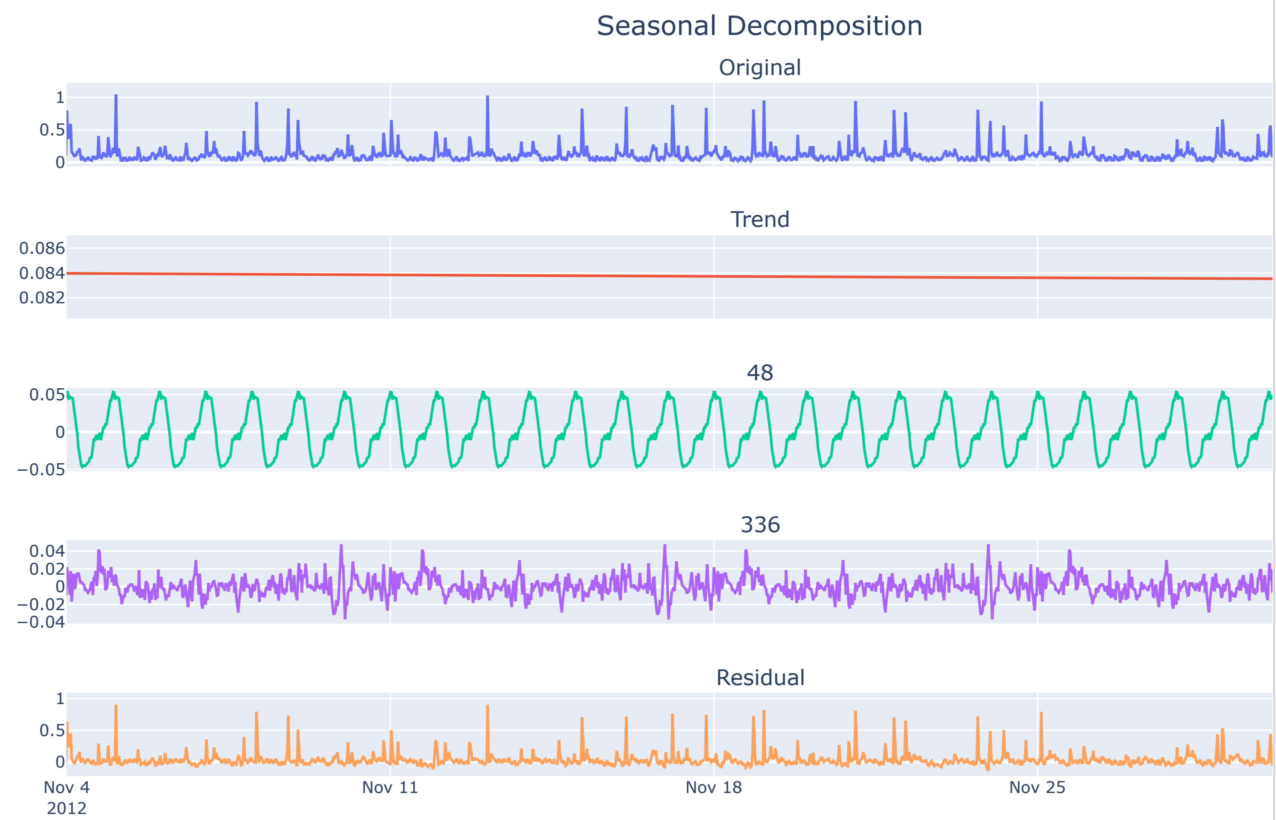
Multiple Seasonality Decomposition using Loess (MSTL)

Using Averages as the seasonal model

```
stl = MultiSeasonalDecomposition(seasonal_model="averages",seasonality_periods=[48*7, 48], model = "additive")
res_new = stl.fit(pd.Series(ts, index=ts_df.index))
```

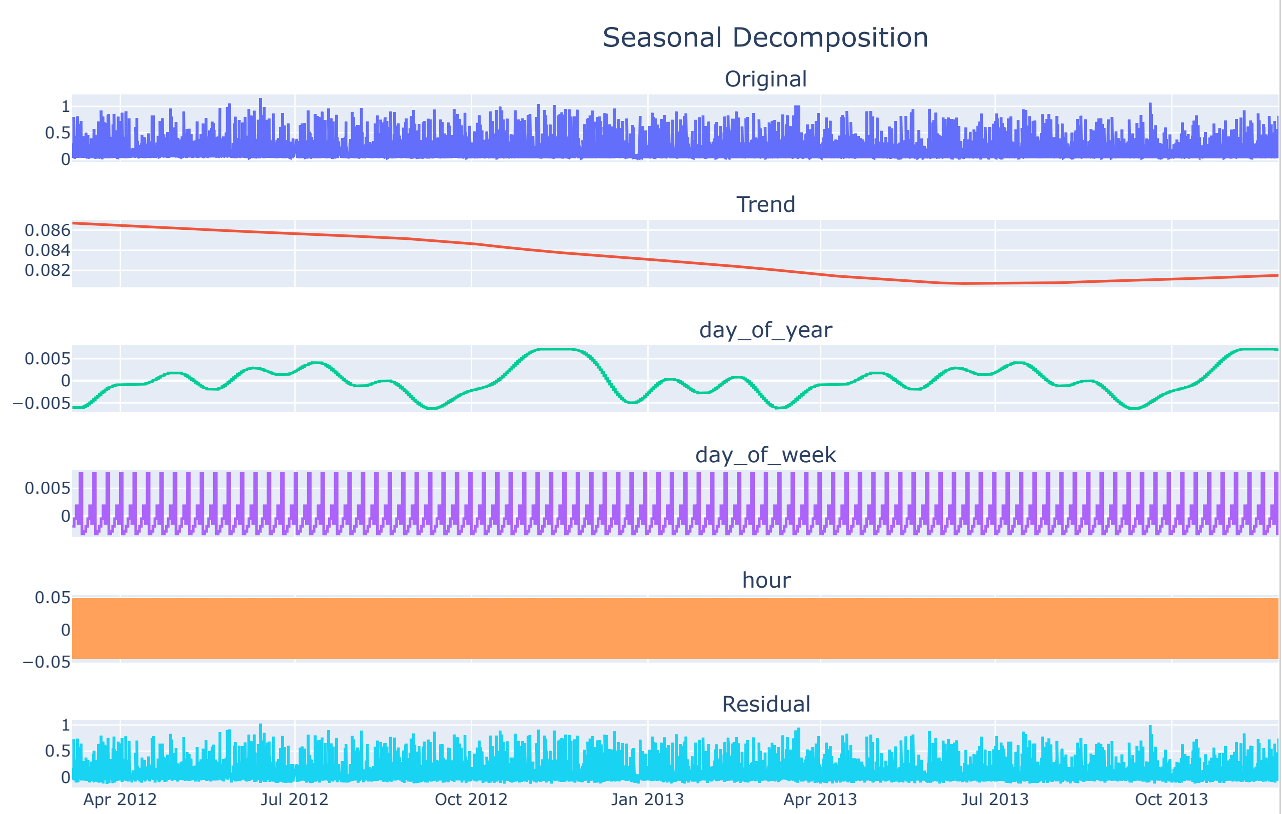



```
fig.update_xaxes(type="date", range=["2012-11-4", "2012-12-4"])
```

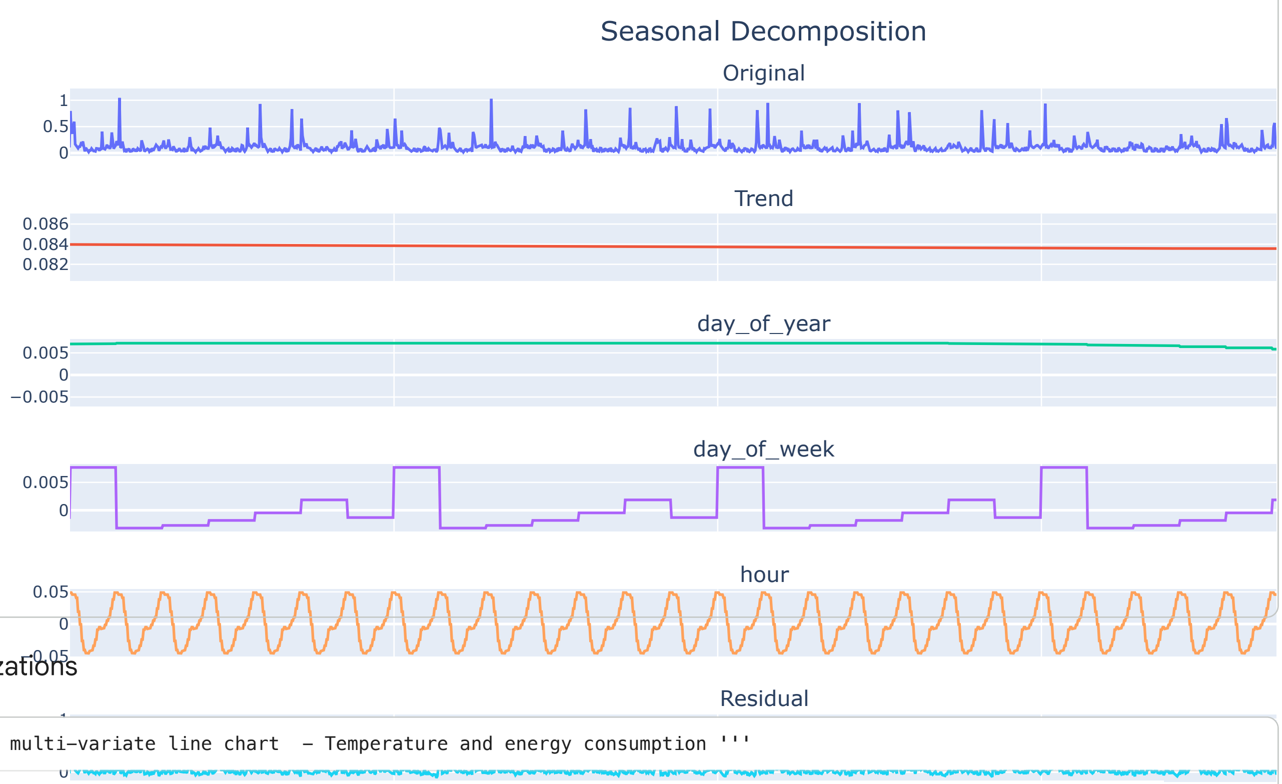


Using Fourier Decomposition as seasonal model

```
stl = MultiSeasonalDecomposition(seasonal_model="fourier",seasonality_periods=["day_of_year", "day_of_week", "hour"]
res_new = stl.fit(pd.Series(ts, index=ts_df.index)
```

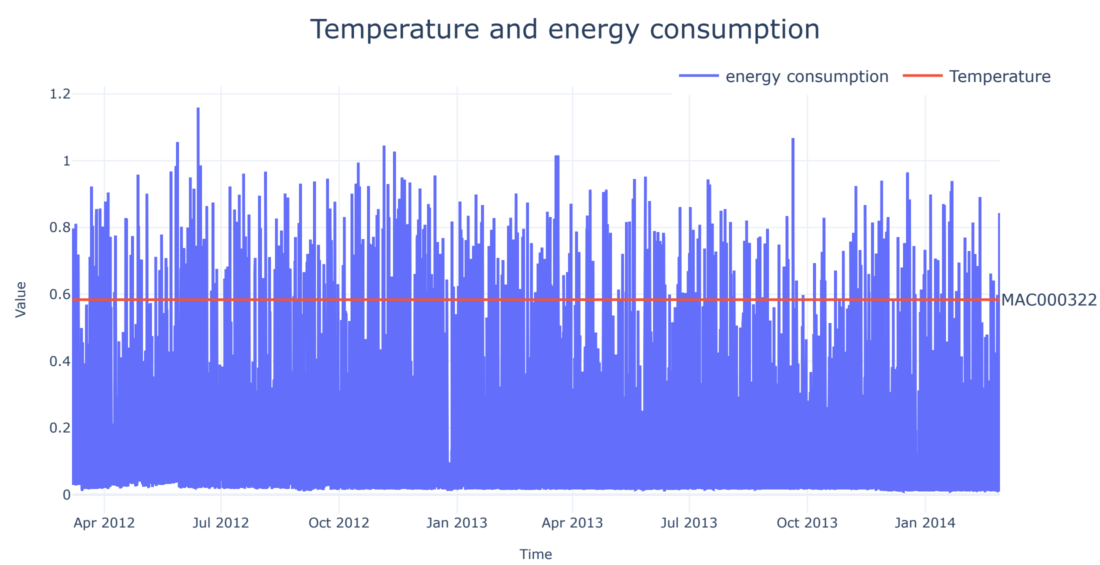



```
#fig.update_xaxes(type="date", range=["2012-11-4", "2012-12-4"])
```

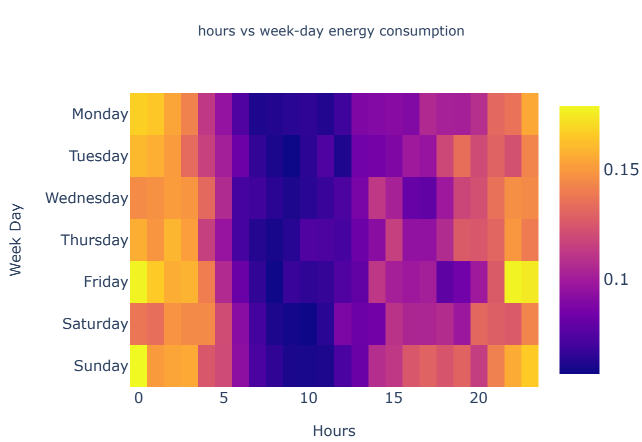


Visualizations

```
''' multi-variate line chart  - Temperature and energy consumption '''
```




```
''' calendar heatmap '''
```




```
''' auto-correlation plots '''
stl = MultiSeasonalDecomposition(seasonal_model='fourier',
  seasonality_periods=['day_of_year', 'day_of_week', 'hour'], model = 'additive',
  n_fourier_terms =10)
res = stl.fit(ts_df['energy_consumption'])
```